Clovertex

Clovertex: Data Engineering Intern Take-Home Assignment

Overview

You are joining a life sciences data platform team. Your objective is to build a local data pipeline that ingests multi-format clinical and genomics data from multiple hospital sites, cleans and unifies it, organizes it into a data lake structure, computes key statistics, and generates analytical visualizations. The entire solution must run inside a Docker container with basic CI/CD on GitHub.

This assignment reflects a real scenario at Clovertex: clinical data arrives from multiple hospital sites in different formats and must be processed reliably before downstream analytics or regulatory reporting.

Duration: 3 days from receipt of this assignment.

Provided Data

Inside the `data/` folder you will find raw clinical data from 3 simulated hospital sites along with genomics data:

File	Format	Description
`site_alpha_patients.csv`	CSV	Patient demographics and admission information from Site Alpha
`site_beta_patients.json`	JSON	Patient demographics from Site Beta
`site_gamma_lab_results.parquet`	Parquet	Lab test results with patient references
`diagnoses_icd10.csv`	CSV	Diagnosis records with ICD-10 codes
`medications_log.json`	JSON	Medication administration log
`clinical_notes_metadata.csv`	CSV	Metadata about clinical notes
`genomics_variants.parquet`	Parquet	Variant call data across oncology genes

`reference/icd10_chapters.csv`	CSV	ICD-10 code to chapter/category mapping
`reference/lab_test_ranges.json`	JSON	Normal reference ranges for lab tests
`reference/gene_reference.json`	JSON	Oncology gene metadata and cancer associations

Tasks

Task 1: Ingest, Clean and Unify

Build a Python pipeline that:

1. Ingests all raw files from `data/`
2. Identifies and resolves data quality issues across all files programmatically
3. Unifies patient records across sites into a single `patients` table with a consistent schema
4. Joins lab results, diagnoses, medications, and genomics data to unified patient records
5. Filters genomics variants to retain only reliable calls. Document your filtering criteria and reasoning.
6. Outputs clean, processed data as Parquet files

Pipeline logging requirement: Your pipeline must output structured JSON stats to stdout for each dataset processed:

```
{
  "dataset": "site_alpha_patients",
  "rows_in": 370,
  "rows_out": 350,
  "issues_found": {
    "duplicates_removed": 20,
    "nulls_handled": 15,
    "encoding_fixed": 7
  },
  "processing_timestamp": "2024-06-15T10:30:00Z"
}
```

Your pipeline must be idempotent. Running it twice should produce identical output.

As part of this task, also produce a `data_quality_report.json` that includes counts of nulls handled, duplicates removed, orphan records found, and schema mismatches resolved, broken down per source file.

Task 2: Data Lake Structure and Metadata

Organize your pipeline output into a local S3-style data lake:

```
datalake/
    raw/              (copy of original files, untouched)
    refined/          (cleaned Parquet outputs from Task 1)
    consumption/      (analytics-ready outputs from Task 3)
```

For each zone, generate a `manifest.json` containing:

- File name, row count, schema (column names and data types)
- Processing timestamp (ISO 8601)
- SHA-256 checksum of each file

In `refined/`, partition `lab_results` using a strategy you consider appropriate for downstream query performance. Document your chosen partitioning approach and the reasoning behind it in your README.

Task 3: Analytics and Anomaly Detection

Using the cleaned data, compute and output to `datalake/consumption/`:

3a. Patient Demographics Summary

- Age distribution, gender split, site distribution
- Save as `patient_summary.parquet`

3b. Lab Result Statistics

- Per test type: mean, median, standard deviation
- Flag values outside reference ranges using the provided reference file
- For `hba1c` and `creatinine`: compute per-patient trend (worsening vs improving across visits)
- Save as `lab_statistics.parquet`

3c. Diagnosis Frequency

- Top 15 ICD-10 chapters by patient count (not diagnosis count)
- Map codes to chapters using the reference file
- Save as `diagnosis_frequency.parquet`

3d. Genomics Variant Hotspots

- Top 5 genes by count of Pathogenic or Likely Pathogenic variants (after filtering unreliable calls)
- For each gene: mean allele frequency, 25th percentile, 75th percentile
- Save as `variant_hotspots.parquet`

3e. Cross-Domain High-Risk Patients

- Identify patients who have:
- At least one `hba1c` reading above 7.0 (diabetic range), AND
- At least one Pathogenic or Likely Pathogenic genomics variant
- Save as `high_risk_patients.parquet`

Note: Some patient IDs in one dataset may not exist in another. There is no single correct answer for how to handle this, but you must handle it deliberately and explain your decision.

3f. Anomaly Detection

- Define what constitutes an anomalous patient record in this clinical context

- Consider: impossible age and diagnosis combinations, lab values incompatible with life, conflicting medication orders, genomics data inconsistencies
- Document your reasoning for what you flag and why
- Save as `anomaly_flags.parquet`

Task 4: Analytical Visualizations

Generate static analytical plots using **matplotlib** or **seaborn** (choose one or combine both). All plots must be saved as PNG files to `datalake/consumption/plots/` and must read exclusively from your pipeline output, not from raw data.

Required plots:

1. **Patient Demographics**: age distribution histogram and gender split bar chart
2. **Diagnosis Frequency**: horizontal bar chart of top 15 ICD-10 chapters by patient count
3. **Lab Results Distribution**: distribution plots for at least 2 test types with reference range boundaries overlaid
4. **Genomics Scatter**: allele frequency vs read depth, colored by clinical significance
5. **High-Risk Summary**: a visualization summarizing the high-risk patient cohort from Task 3e
6. **Data Quality Overview**: a chart summarizing pipeline quality metrics (nulls handled, duplicates removed, etc.)

Each plot must have a clear title, axis labels, and legend where applicable. Include a brief `plots_README.md` in the plots directory explaining each visualization.

Task 5: Containerization and CI/CD

5a. Docker

Provide a `Dockerfile` and a `docker-compose.yml` that containerizes your entire pipeline.

Requirements:

- `docker-compose up` runs the full pipeline (Tasks 1 through 4) end-to-end in a single command
- The container must exit with code 0 only if all tasks complete without errors
- All outputs (datalake/) should be written to a mounted volume so they persist after the container exits
- Multi-stage build is preferred but not required

5b. GitHub Actions CI

Create `.github/workflows/ci.yml` that runs on every push:

1. **Lint**: run `ruff` or `flake8` on all Python code; fail the workflow on any errors
2. **Docker build check**: verify the container image builds successfully

Your CI must be green on the `main` branch. Submissions with a failing CI will not be reviewed.

Bonus: AI/LLM Classification Task (Optional)

If time permits, the `clinical_notes_metadata.csv` file contains a `note_category` column with free-text descriptions of clinical note types.

Build a classifier that maps each free-text `note_category` into one of these standardized categories:

`["Admission", "Discharge", "Progress", "Surgical", "Consultation", "Lab Review", "Other"]`

- Output the mapping as a new column in your processed data
- Evaluate your classifier: report accuracy against the `expected_category` column
- Analyze failures: discuss where it fails and why

This can be regex/rule-based, LLM API-based, or use a local model. If you do not have API access, a rule-based approach with clear documentation is acceptable.

Expected Repository Structure

```
your-repo/
    README.md
    Dockerfile
    docker-compose.yml
    requirements.txt
    pipeline/
        main.py
        ingestion/
        cleaning/
        transformation/
        stats/
        utils/
    data/
        reference/
    datalake/
        raw/
        refined/
        consumption/
            plots/
    .github/
        workflows/
            ci.yml
    .gitignore
    .dockerignore
```

Documentation Requirements

Your README must include:

1. Setup instructions: how to run everything locally with Docker
2. Pipeline architecture: each stage and why you structured it that way
3. Data cleaning decisions: what issues you found and how you resolved them, with reasoning
4. Data lake design: partitioning rationale, zone purpose, manifest structure
5. Anomaly detection logic: what you defined as anomalous and your domain reasoning
6. Genomics filtering criteria: what you considered reliable and why
7. Assumptions: anything you assumed that was not explicitly stated

8. Improvements: what you would change with more time or for a production deployment

---

Evaluation Criteria

- Pipeline correctness: does the data come out clean, unified, and accurate?
- Code quality: modular, readable, well-structured Python
- Data engineering judgment: how you handle ambiguity in the data
- Data lake design: sensible zone structure, manifests, partitioning rationale
- DevOps competence: Docker works on first run, CI is green
- Domain awareness: do your anomaly rules and cleaning decisions reflect clinical understanding?
- Documentation quality: can someone else understand and reproduce your work?
- Visualization quality: are the plots clear, accurate, and well-labeled?

We value thoughtful decisions over perfect code. If you make a tradeoff, explain it.

Rules

- Any Python library available on PyPI is permitted
- No cloud account is required. Docker only, runs entirely locally.
- Use relative paths throughout. No hardcoded absolute paths.
- AI coding tools are permitted for boilerplate, but you must be able to explain every line of your code in a follow-up interview
- If you get stuck on a task, document your approach and move on. Partial credit is given for well-reasoned incomplete work.

Submission

1. Push your completed work to a public GitHub repository
2. Ensure your CI badge is green on `main`
3. Email with:

- Subject: `DE-Assignment - [Your Full Name]`
- Your GitHub repository link
- A 2-3 sentence note on the most interesting decision you made

Deadline: 3 days from the date you receive this assignment.

Pre-submission checklist:

- `docker-compose up` works end-to-end
- GitHub Actions CI passes on `main`
- All plots are generated in `datalake/consumption/plots/`

- README covers all documentation requirements
- `datalake/` directory is gitignored

Clovertex Engineering Team